

AI usage log template

Task 1.SCRUM-24: Build a pipeline

Purpose of AI Use: To help define a simple pipeline to follow for testing code.

Prompt: how to design a simple pipeline testing for a web-based app

Response:

To design a simple, robust testing pipeline for a web-based application, you need to establish an **automated quality gate** that executes tests sequentially from fastest to slowest. The core strategy relies on a **"Shift-Left" approach**, running lightweight syntax and unit checks first, and only executing heavy browser simulation if the initial phases pass.

1. The Core Anatomy of the Pipeline

A reliable, simple testing pipeline is split into **four distinct stages** executed every time a developer creates a Pull Request (PR) or pushes code to the repository:

[Code Push] —> 1. Lint & Static Analysis —> 2. Unit Testing —> 3. API / Integration —> 4. E2E (Browser) Testing

2. Step-by-Step Stage Configuration

Stage 1: Code Quality & Linting (Seconds)

Before executing any logic, verify that the code complies with structural and security standards.

- **What it does:** Catches syntax errors, code formatting discrepancies, and obvious security vulnerabilities without executing the software.
- **Tools to use:** ESLint or Biome for JavaScript/TypeScript, Flake8 or Black for Python.

Stage 2: Unit Testing (Under 1 Minute)

Isolate individual functions, components, or UI utility logic to ensure they return exact expected outputs.

- **What it does:** Tests pure functions, mathematical utilities, form validators, or isolated frontend components. Mock all network requests and database operations here to keep execution instantaneous.
- **Tools to use:** Vitest or Jest (Frontend), pytest (Python Backend).

Stage 3: API & Integration Testing (1–3 Minutes)

Verify that your application components communicate smoothly with adjacent systems, databases, and network responses.

- **What it does:** Tests real backend HTTP endpoints, authentication middleware, database queries, or external service integrations.
- **Tools to use:** Supertest (Node.js), Postman/Newman, or native framework test clients.

Stage 4: End-to-End (E2E) Browser Testing (3–5 Minutes)

Spin up a headless browser environment to simulate actual user journeys.

- **What it does:** Targets **5 to 7 critical user paths** (e.g., *User Login, Add to Cart, Payment Checkout*). Do not attempt to test every edge case here; E2E tests are slower and more prone to network flakiness.
- **Tools to use:** [Playwright](#) (highly recommended for modern web apps) or [Cypress](#).

 3. Direct Tool Comparison

For a simple startup setup, choose **GitHub Actions** if your code is already on GitHub—it requires zero infrastructure management. Choose **GitLab CI** or **Jenkins** if you prefer self-hosted or dedicated control.

Feature / Criteria	GitHub Actions	GitLab CI/CD	Jenkins
Hosting Model	Cloud-managed (SaaS)	Cloud-managed or Self-hosted	Fully Self-hosted
Configuration	YAML (.github/workflows/)	YAML (.gitlab-ci.yml)	Groovy script (Jenkinsfile)
Complexity	Very Low (Plug & play)	Low (Built into platform)	High (Requires server setup)
Best For	Fast setups & open source	All-in-one dev environments	Custom enterprise infrastructure



4. Example: A Simple GitHub Actions Pipeline

Save this configuration file as `.github/workflows/test-pipeline.yml` in your project repository to immediately establish your pipeline:

```
name: Web App Testing Pipeline

on:
  push:
    branches: [ main, dev ]
  pull_request:
    branches: [ main ]

jobs:
  test-suite:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository code
        uses: actions/checkout@v4

      - name: Setup Node.js Environment
        uses: actions/setup-node@v4
        with:
          node-version: '20'
          cache: 'npm'

      - name: Install Project Dependencies
        run: npm ci

      - name: Run Stage 1 (Linting)
        run: npm run lint

      - name: Run Stage 2 (Unit Tests)
        run: npm run test:unit

      - name: Run Stage 3 (Integration/API Tests)
        run: npm run test:integration

      - name: Install Playwright Browsers (For Stage 4)
        run: npx playwright install --with-deps

      - name: Run Stage 4 (End-to-End Tests)
        run: npm run test:e2e
```

AI-Suggested Content: Generated a pipeline that consists of 4 stages, described each stage and gave a configuration file to establish the pipeline in the repository.

Validation: Compared the pipeline with other proposed pipelines.

Decision: Partially accepted as the configuration has yet to be tested

Reflection: AI helped define a pipeline that is appropriate to follow for a web-based application. AI also assisted in identifying the most appropriate tool for constructing the pipeline.

Responsible Person: Johnny Quach